

Typed Decisions in Agent Memory: Where They Help, Where They Don't, and What It Costs

Rishabh Sharma*

Independent Researcher

rishabh.sharma1103@gmail.com

Abstract

In engram, an agent memory system, an LLM extracts facts and every later decision is a typed question answered by a hosted decision model (Jev). This makes re-examining the store affordable: engram writes at about the cost of add-only mem0 2.1.0, which never revisits what it stored (\$9.90 against \$9.80 per 1,000 messages on the dev slice, \$10.22–\$10.55 against \$9.92–\$10.16 held out). Against an LLM decider using the same extraction, our claude-sonnet-4-6 implementation of mem0's update prompt, typed decisions had 70.0× lower decision cost and 27.6× lower median decision latency (that arm made no escalations), and equal accuracy on a 35-question dev slice. On 610 held-out LoCoMo questions, at a matched mean retrieved-context budget, engram was +8.7 points above mem0 (95% CI +5.2 to +12.1); with Jev's reranking turned off it was 13.4 points lower, so the reranker accounts for the whole difference. At $k=20$ the two systems are indistinguishable (+0.8, CI -2.3 to +3.9). On update sets drafted with an AI assistant, 0/8 no-close trap items were closed, and closing stale facts did not change answers.

1 Introduction

The cost problem. A memory system for an LLM agent turns conversation into facts it can retrieve later. Each write has two parts. Extraction asks what facts a message states. Decisions ask, for each fact, whether it is new, a duplicate, a refinement or a change to something already stored, whether it is worth keeping, and whether it is sensitive. In current open-source systems both parts are LLM calls. mem0 2.1.0's default add() makes a single LLM call per message with its additive extraction prompt, and has no separate update or delete step (mem0/memory/main.py, Memory._add_to_

vector_store). An LLM call per decision is expensive enough that nothing re-examines the store afterwards. Facts that stopped being true stay in it.

The observation. The decisions are choices among options fixed in advance. That is the setting typed decision models are built for. They return a probability over a fixed option set in one short request, and they cost far less than generating text. engram is built on that observation: an LLM extracts facts, and every decision after extraction is a typed question (Figure 1).

Concurrent work. The architectural idea of routing memory decisions to a typed decision model was reached independently by Jev-Mem (Jiang et al., 2026a), which uses Jev for typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. In its released code the type scores are computed but not used downstream, and Jev supplies routing probabilities from which code allocates the traversal budgets. It reports a LoCoMo judge score of 0.777 with a 158 s build and 0.93 s query against a Full Context baseline, A-MEM (Xu et al., 2025), MAGMA (Jiang et al., 2026b) and two further systems reported in that paper; that score is a continuous partial-credit judge score with adversarial questions scored by string match, so it is not on our accuracy scale. It does not ablate the decision layer with extraction and node representation held fixed (its no-Jev ablations fall back to MAGMA's LLM-extraction pipeline), evaluate updates or closes, measure calibration, or use a held-out split or confidence intervals. Separately, a community article reranked AtMem's top-10 with Jev on 1,986 LoCoMo questions and measured the effect at the ranking level: MRR@5 rose from 0.4259 to 0.5868 and Recall@1 from 0.3399 to 0.5423 (Taghia, 2026). This paper's contribution is the controlled measurement: an ablation of the decision layer using the same extraction model, prompt construction and implementation, its calibration,

*Preprint (v1.1). DOI: [10.5281/zenodo.22948964](https://doi.org/10.5281/zenodo.22948964). Version 1: [10.5281/zenodo.22941758](https://doi.org/10.5281/zenodo.22941758).

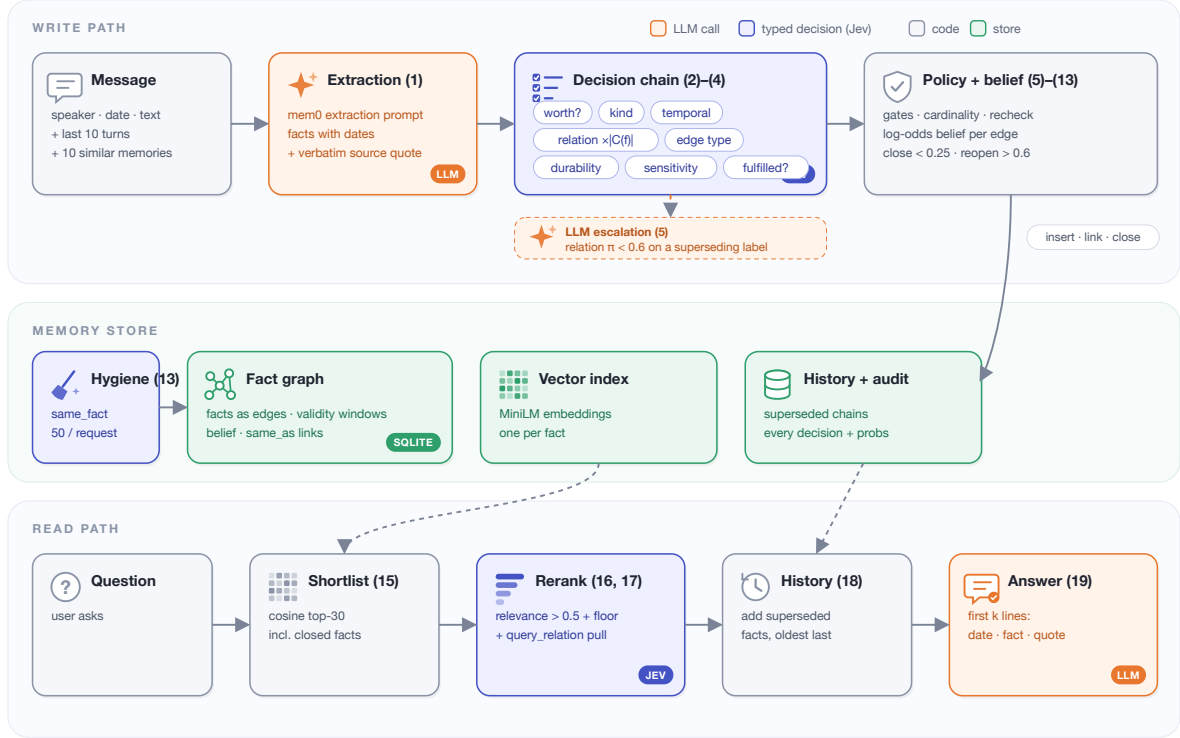


Figure 1: engram’s write and read paths. Only extraction, the answer and the escalation of unsure superseding relations are LLM calls (orange); every other decision is a typed Jev question (blue).

the behaviour of the store it drives, the retrieval effect on answers with a matched-context control and a held-out no-rerank arm, and a negative result.

Contributions. The paper makes three. First, typed decisions make re-examining the store affordable: engram writes at about add-only mem0’s cost (\$9.90 against \$9.80 per 1,000 messages on the dev slice, \$10.22 to \$10.55 against \$9.92 to \$10.16 held out). Against an LLM decider using the same extraction model, prompt construction and implementation, our claude-sonnet-4-6 implementation of mem0’s update prompt with one call per extracted fact, typed decisions had $70.0\times$ lower decision cost and $27.6\times$ lower median decision latency, with equal accuracy on a 35-question dev slice (§5.1). The latency ratio excludes escalations: the Jev arm it measures made 0. The frozen system escalates; on the held-out conversations its median decision latency was 868 ms to 1,982 ms, and its median end-to-end write was slower than mem0’s on 3 of 4. Second, a belief-state store policy with reversible, gated closes closed 0/8 no-close trap items, with 1 wrong plan_fulfilled close and 1 close matching no labeled pair, and its v3 rule removes a weak-evidence failure that closes true facts (§3.2, §5.3). Third, on 610 held-out questions at a matched mean

retrieved-context budget, engram was +8.7 points above mem0; turning Jev’s reranking off costs 13.4 points on the same questions and leaves engram at -4.8 points against token-matched mem0, so on held-out data the reranker accounts for the whole difference (§5.2). A negative result frames all three: closing stale facts did not change answers (§5.4). Figure 2 plots the third result against the context each system shows the answer model.

Scope. We compare against one baseline (mem0 OSS 2.1.0) on one benchmark (LoCoMo: one development conversation and four held-out conversations, scoring four of its five question categories; adversarial is excluded from the primary comparison, following mem0’s evaluation protocol, and reported separately in Table 3). We use update sets drafted and labeled with an AI assistant at the author’s direction (§4.3), and one hosted decision model (Jev, jev-1.13.0). We did not test Zep/Graphiti or Letta, LongMemEval, other extraction, answer or judge models, multi-user stores, or non-English text. Every finding, the negative result in particular, is about these LoCoMo-derived evaluations with this extraction, rendering and answer setup.

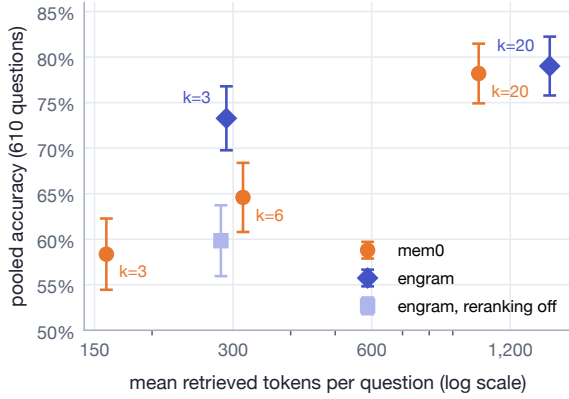


Figure 2: Pooled held-out accuracy (610 questions) against $T(k)$, with per-question 95% intervals; models as in Table 2. No line joins the points: mem0’s accuracy was measured at $k=3, 6$ and 20 only, and the token-matching sweep counted tokens at the other k without answering. engram at $k=3$ sits above mem0 at $k=6$, which shows the answer model slightly more tokens; with reranking off, engram falls below mem0 at the same budget; at $k=20$ the two systems are indistinguishable.

2 Background and Related Work

2.1 Write decisions in existing systems

mem0 (Chhikara et al., 2025) extracts and consolidates facts with LLM calls. Version 2.1.0 extracts with one LLM call per message; the prompt receives the new message, the last 10 messages and the 10 most similar existing memories, and emits only additions. Its older update step, which asked an LLM to label each new fact as ADD, UPDATE, DELETE or NONE against existing memories, is still shipped as DEFAULT_UPDATE_MEMORY_PROMPT. We use it as the LLM decision layer in §5.1.

Other systems also decide with an LLM. Zep’s Graphiti builds a temporal knowledge graph and resolves entities and invalidates edges with LLM calls (Rasmussen et al., 2025). MemGPT, now Letta, manages memory tiers through LLM function calls (Packer et al., 2023). A-MEM links notes with LLM-written attributes (Xu et al., 2025), and MAGMA organizes memory as multiple graphs (Jiang et al., 2026b). ByteRover curates a hierarchical context with an LLM; its retrieval is a five-tier progressive strategy that answers most queries in under 100 ms without LLM calls and escalates to agentic reasoning only for novel questions (Nguyen et al., 2026).

2.2 Typed decision models

A typed decision model takes a shared *state* (JSON) and a set of questions, and returns a probability dis-

tribution for each question. A question is a choice among named options, each with a rubric; a yes/no (“noul”) question; or a score.

Jev. We use Jev through TypeSafe’s API: model jev-1.13.0, priced at 0.042 USD per million input tokens as billed to our account (src/engram/config.py); the public documentation does not list a price. Its documentation gives a limit of 255 options per choice question (TypeSafe AI, 2026). Its latency varies modestly with request size (§5.5).

An open-weights model with the same request format, Laya, was tested zero-shot as an exploratory alternative; it is reported in Appendix D.

2.3 LoCoMo and its limits

LoCoMo (Maharana et al., 2024) contains long multi-session conversations with questions in five categories, of which we score four (adversarial is excluded from the primary comparison, following mem0’s evaluation protocol; §4.1). LongMemEval (Wu et al., 2025) also evaluates long-term conversational memory; we used only LoCoMo.

It barely tests updates. In the first 215 messages of conv-26, an LLM labeler (claude-sonnet-4-6, prompt in bench/stale.py) found 2 claims that a later message makes no longer true (bench/slices/conv26_superseded.json).

Public scores are not comparable. Published LoCoMo scores for the same systems differ between the systems’ own reports and third-party reports (Mem0, 2026; ByteRover, 2026). We do not quote those numbers. All comparisons here run both systems under one protocol.

2.4 Closest work and adjacent ideas

Jev-Mem and AtMem. Jev-Mem (Jiang et al., 2026a) is the closest design: Jev controls typing, relation construction, query routing, budget allocation, traversal, candidate scoring and stopping. The two designs differ in the store and in retrieval. Jev-Mem stores raw conversation turns and uses no LLM on successful writes, so its write path avoids the extraction cost that dominates engram’s and mem0’s (§5.1). It runs with admission filtering off and preserves every observation, so its store is never updated or closed; engram’s store is governed by a close/belief policy with reversible closes and merges (§3.2). Jev-Mem routes queries across multiple views; engram uses a single listwise rerank over cosine candidates plus a query-relation pull (§3.3). Its released benchmark runner (commit 81574eb) selects the retrieval depth and the answer

prompt from the gold question category and by default keeps the best of three answers as scored against the gold answer; its README advises `--best-of-n 1`. The AtMem-Jev article (Taghia, 2026) measured the rerank at the ranking level (Recall@10 unchanged; median batch latency 3.32 s) and reported no answer accuracy or intervals.

Reranking and context. Long contexts are used poorly by language models (Liu et al., 2024), and LLMs rerank candidates well (Sun et al., 2023); spending compute on selecting context rather than adding more of it follows from both, and our rerank result is an instance.

Routers. Routers send queries to cheaper models (Ong et al., 2025; Chen et al., 2024), and small classifiers act as guardrails (Inan et al., 2023). The escalation rule Eq. (5) is a router of that kind.

Calibration. Temperature scaling (Guo et al., 2017) and conformal prediction (Angelopoulos and Bates, 2021) give principled thresholds for acting on a model’s probability. The belief update Eq. (10) uses probabilities as likelihoods, so it depends on calibration, which §5.3 measures.

3 engram

3.1 Setup and notation

A conversation is a sequence of messages $m_1, m_2, \dots$, and $\text{date}(m_t)$ is the time message m_t was said. After message t the store is $M_t = (F_t, V_t, I_t)$. F_t is the set of facts, V_t the entity nodes, and I_t a vector index over fact texts. A fact $u \in F_t$ is an edge from its subject $\text{subj}(u) \in V_t$ to an object node, labelled with a relation $\text{rel}(u)$, and carries its text, the verbatim source quote, date of its message, a validity window whose end $\text{valid_until}(u)$ is empty while the fact holds, and a belief $b_u \in [b_{\min}, b_{\max}]$ that it is currently true, with $b_{\min} = 0.02$ and $b_{\max} = 0.98$. $\text{ent}(u)$ is the set of named entities in its text, and $\text{card}(\rho) \in \{\text{one}, \text{many}\}$ says whether relation type ρ is single- or multi-valued. $s_{\cos}(x, y)$ is the cosine similarity of their MiniLM embeddings in I_t , and $\text{Top}_n s_{\cos}(x, \cdot)$ is the list of the n facts most similar to x , most similar first.

A decision backend $D \in \{\text{Jev}, \text{LLM}\}$ answers typed questions. A typed question Q has an option set O_Q ; given state s , $P_D(o \mid s, Q)$ is the probability D gives option $o \in O_Q$. Its decision is the argmax option and its confidence π the largest probability. X is the extraction call (an LLM with mem0’s prompt) and L an LLM call (escalation

or answer). Five thresholds act on these probabilities, all read from `src/engram/config.py` and checked against this paper by `tests/test_paper_thresholds.py`: a decision acts at $\theta_{\text{act}} = 0.85$, a superseding relation below $\theta_{\text{esc}} = 0.60$ is escalated, a retrieved fact is relevant above $\theta_{\text{rel}} = 0.5$, and belief closes an edge below $\theta_{\text{close}} = 0.25$ and reopens it above $\theta_{\text{open}} = 0.60$. τ_Q is a per-question temperature, which rescales probabilities as P^{1/τ_Q} renormalised; it is fitted to measure calibration (§5.3) and is not applied on the write path.

On the read path, q is a question, k the retrieval budget (the answer model sees k memory lines), and $T(k)$ the mean number of retrieved-context tokens per question at budget k . Two systems answer the same questions, so every comparison is paired: for question i , $d_i = \text{engram}_i - \text{mem0}_i$ with each term 1 if the judge marks the answer correct and 0 otherwise.

3.2 Write path: decision chain and policy layer

Figure 1 shows both paths, with the equations below marked on its boxes. For message m_t :

$$F_t^{\text{new}} = X(m_t, m_{t-10}, \dots, m_{t-1}, \text{Top}_{10} s_{\cos}(m_t, \cdot)) \quad (1)$$

Extraction sees the new message, the last ten messages and the ten most similar stored facts, as mem0 2.1.0’s `add()` does, so both systems extract from the same kind of prompt.

$$\begin{aligned} C(f) = & \text{Top}_{10} s_{\cos}(f, \cdot) \\ & \cup \text{Top}_{10} \{u : \text{subj}(u) = \text{subj}(f) \\ & \quad \wedge \text{rel}(u) = \text{rel}(f) \\ & \quad \vee \text{ent}(u) \cap \text{ent}(f) \neq \emptyset\} \end{aligned} \quad (2)$$

The graph half finds stored facts about the same subject and relation, or sharing a rare named entity, that cosine similarity ranks too low; each half is capped at ten, closest first.

$$\begin{aligned} d_j(f) &= \arg \max_{o \in O_j} P_D(o \mid f, Q_j), \\ \pi_j(f) &= \max_{o \in O_j} P_D(o \mid f, Q_j) \end{aligned} \quad (3)$$

The fact questions (worth remembering, kind, temporal status, relation type, durability, sensitivity)

go to D in one request with the relation questions of the next equation, so a fact costs one call.

$$\begin{aligned} r(f, u) &= \arg \max_{o \in R} P_D(o \mid f, u, Q_{\text{rel}}), \\ \pi(f, u) &= \max_{o \in R} P_D(o \mid f, u, Q_{\text{rel}}), \\ u^* &= \arg \max_{u \in C(f)} \max_{o \in R \setminus \{\text{new}\}} P_D(o \mid f, u, Q_{\text{rel}}) \end{aligned} \quad (4)$$

Here $R = \{\text{new, duplicate, refinement, update, contradiction, negates}\}$ and $S = \{\text{update, contradiction, negates}\}$ is the superseding subset; the write decision is about the candidate u^* the new fact most likely relates to, with relation $r^* = r(f, u^*)$ and confidence π^* its largest non-new probability.

$$\begin{aligned} &\text{act if } \pi^* \geq \theta_{\text{act}}; \\ &r^* \leftarrow L(f, u^*) \text{ if } r^* \in S \wedge \pi^* < \theta_{\text{esc}}; \\ &\text{tentative otherwise, or if } \pi_{\text{worth}}(f) < \theta_{\text{act}} \end{aligned} \quad (5)$$

Only the uncertain superseding decisions, the ones that could wrongly close a fact, pay for an LLM call.

$$\begin{aligned} g_T(f) &= \mathbb{K}[P_D(\text{current} \mid f) \\ &\quad + P_D(\text{past} \mid f) \geq \theta_{\text{act}}] \end{aligned} \quad (6)$$

A planned or hypothetical statement never counts against a stored fact, and the gate sums current and past because Jev splits a completed change between them.

$$\begin{aligned} g_C(f, u) &= \mathbb{K}[r = \text{negates}] \\ &\vee \mathbb{K}[r = \text{update} \\ &\quad \wedge (\text{card}(\text{rel}(u)) = \text{one} \vee \text{sib}(f, u))] \\ &\vee \mathbb{K}[r = \text{contradiction} \wedge \text{card}(\text{rel}(u)) = \text{one} \\ &\quad \wedge \text{sib}(f, u)] \end{aligned} \quad (7)$$

With $r = r(f, u)$ and $\text{sib}(f, u)$ meaning same subject and relation, a new value can replace an old one only where the relation holds one value at a time, so “likes hiking” does not close “likes painting”.

$$g_A(f, u) = \mathbb{K}[r'(f, u) \in S \wedge \pi'(f, u) \geq \theta_{\text{act}}] \quad (8)$$

The first piece of evidence against a fact, and every update on a multi-valued relation, must be confirmed by a second phrasing of the relation question (r' , π'), so one misread answer cannot start a close.

$$\begin{aligned} e(f, u) &= +\text{logit } \pi(f, u) \\ &\quad \text{if } r \in \{\text{duplicate, refinement}\}; \\ e(f, u) &= -\text{logit } \pi(f, u) - \text{logit } \pi'(f, u) \\ &\quad \text{if } r \in S \wedge g_T g_C g_A = 1; \\ e(f, u) &= 0 \text{ otherwise} \end{aligned} \quad (9)$$

Answers become evidence in log-odds, and the recheck term appears only when the recheck was asked.

$$\begin{aligned} \text{logit } b_u &\leftarrow \text{logit } b_u + e(f, u), \\ \text{logit } b_f &\leftarrow \text{logit } b_f - \sum_u \min(e(f, u), 0) \end{aligned} \quad (10)$$

Nothing irreversible happens on one answer: evidence accumulates on the stored fact, clipped to $[b_{\min}, b_{\max}]$, and the new fact, which starts at 0.5 if tentative and at π^* otherwise, gains what the old one loses. In v2 (the held-out runs) every answer counts; v3 counts an answer only when $\pi(f, u) > 0.5$, so weak support cannot lower belief.

$$\begin{aligned} \text{valid_until}(u) &\leftarrow \text{date}(m_t) \text{ if } b_u < \theta_{\text{close}}, \\ \text{valid_until}(u) &\leftarrow \emptyset \text{ if closed by belief} \\ &\text{and } b_u > \theta_{\text{open}} \end{aligned} \quad (11)$$

The gap between the two thresholds keeps a fact from flickering open and closed on alternating answers.

$$\begin{aligned} \text{valid_until}(u) &\leftarrow \text{date}(m_t) \text{ if} \\ &P_D(\text{yes} \mid f, u, Q_{\text{ful}}) \geq \theta_{\text{act}} \\ &\wedge (\text{rel}(u) \in \{\text{plans, goal}\} \\ &\quad \vee d_{\text{temporal}}(u) = \text{planned}) \end{aligned} \quad (12)$$

A plan ends when it happens, which the relation question does not ask, so a dedicated yes/no question decides it.

$$\begin{aligned} &\text{same_as}(f, u^*) \text{ if } r^* = \text{duplicate} \\ &\quad \wedge \pi^* \geq \theta_{\text{act}} \wedge f \text{ not tentative;} \\ &\text{hygiene: link}(u, v) \text{ if} \\ &\quad P_D(\text{duplicate} \mid u, v, Q_{\text{same}}) \geq \theta_{\text{act}}, \\ &\text{unlink if } \leq 1 - \theta_{\text{act}} \end{aligned} \quad (13)$$



Figure 3: Belief in the fact “Melanie carves out daily me-time through running, reading, or playing violin” (message D2:5), dev + update set 1, under v2 and v3. Source: the belief trace in `bench/results/e4_belief_v2_dev_updates_k3_noanswer.json` and its v3 counterpart. Under v2 the refinement answer at D2:7 ($\pi = 0.45$) lowers belief slightly, so the update at U:E11 takes it below θ_{close} ; v3 ignores that answer (×) and the fact closes one message later, at U:S01.

Duplicates are linked, never merged, so a wrong merge can be undone; after ingestion one hygiene pass re-asks every pair that shares subject and relation (the 30 most recent per group, 50 pairs per request).

$$M_{t+1} = U(M_t, F_t^{\text{new}}, \{d_j(f), r(f, u)\}) \quad (14)$$

U applies the equations above in order, plus two overrides decided by rule, not by D : an explicit request to remember sets $\pi_{\text{worth}} = 1$, and a credential is redacted before any decision or storage.

On the held-out conversations one hygiene pass made between 1,865 and 5,446 decisions, for between \$0.0112 and \$0.0332 (§5.3). With LLM decisions, re-examining the store at that scale is what becomes unaffordable.

Figure 3 traces one fact through Eqs. (10) and (11) under both rules.

3.3 Read path

For question q and budget k :

$$A(q) = \text{Top}_{30} s_{\text{cos}}(q, \cdot) \quad (15)$$

The shortlist includes closed facts, so a question about the past can reach them.

$$K(q) = [u \in A(q) : P_D(\text{yes} \mid u, q, Q_{\text{rlv}}) > \theta_{\text{rel}}] \\ \text{by } P_D(\text{yes} \mid u, q, Q_{\text{rlv}}) b_u \text{ descending,} \\ \text{then the first ten of } A(q) \text{ not in } K(q) \quad (16)$$

One request scores every shortlisted fact, and the cosine floor keeps ten facts even when Jev judges few relevant.

$$g(q) = \arg \max_g P_D(g \mid q, Q_{\text{qr}});$$

$$K(q) \leftarrow K(q) \cup \{u : \text{rel}(u) = g(q)\} \\ \text{if } g(q) \neq \text{none} \wedge \max_g P_D(g \mid q, Q_{\text{qr}}) \geq \theta_{\text{act}} \quad (17)$$

A question that names a relation (“where does she live?”) pulls in up to 10 currently valid facts of that relation, which cosine similarity to the question may miss.

$$K^+(q) = K(q) \cup \bigcup_{u \in K(q)} \text{chain}(u) \cup N(K(q)) \quad (18)$$

$\text{chain}(u)$ is the facts u superseded (linked through `valid_until`), so “before Berlin” finds Paris; N adds up to 15 facts one hop from the kept facts’ objects, skipping nodes with more than 25 edges; a `same_as` cluster is shown once.

$$a = L(q, \text{render}(\text{first } k \text{ of } K^+(q))) \quad (19)$$

Each line is the date the fact was said, the fact and its source quote, so the answer model can resolve recency itself. With reranking off, $K^+(q) = A(q)$ followed by its chains, so at $k = 3$ the answer model sees the cosine top three.

3.4 Cost model

With a per-decision Jev cost c_J , an escalation cost c_L and an escalation threshold θ on π :

$$C(\theta) = c_J + P(\pi < \theta) c_L \quad (20)$$

Every decision pays for Jev, and only the escalated share pays for the LLM.

$$E(\theta) = P(\pi \geq \theta) \varepsilon_J(\theta) + P(\pi < \theta) \varepsilon_L \quad (21)$$

$\varepsilon_J(\theta)$ is Jev’s error rate on the decisions it keeps and ε_L the LLM’s on the escalated ones, so raising θ trades cost for error only if $\varepsilon_L < \varepsilon_J$; §5.3 draws both curves.

4 Experimental Setup

4.1 Models, data and protocol

Models. Every arm uses the same models. `claude-haiku-4-5` extracts facts, for `engram` and for `mem0 2.1.0` (which runs with telemetry off). `claude-sonnet-4-6` answers and judges, with `mem0`’s LoCoMo evaluation prompts (§4.2), and is also the LLM decision layer and the escalation model, with `mem0`’s update prompt.

Data. The *dev* slice is conv-26 sessions 1–4 (76 messages, 35 questions), and the *stress* slice is conv-26 sessions 1–10 (215 messages, 80 questions). The *held-out* slice is conv-30, conv-41, conv-42 and conv-43 whole (610 questions); none of it was used during development, and the system configuration was frozen (git tag `e4-frozen`) before any held-out run. LoCoMo’s questions fall in five categories, of which we score four (adversarial is excluded from the primary comparison, following `mem0`’s evaluation protocol). Each held-out conversation is ingested once per system, and $k=3$ and $k=20$ are answered from the same store.

Caching and budget. Every LLM and Jev call is cached by its full request, and budget guards stop any run past a spending limit. Phase 2 (all experiments reported here) spent \$101.79 over 84 ledgered runs. Jev accounts for \$1.51 of it (`bench/results/phase2_spend.jsonl`; per-arm totals in Appendix E). The build phase was not ledgered; roughly \$10 by the author’s estimate.

4.2 Shared extraction

For each message, `mem0 2.1.0`’s `add()` builds its extraction prompt from the new message (as `[date] speaker: text`), the last 10 messages, the 10 existing memories most similar to it, and a current date. It passes no observation date, so relative dates in historical conversations resolve against the current date. We report this as-is, and a patched variant in §6.

`engram`’s E-arms build the same prompt with the same function (`generate_additive_extraction_prompt`), inputs and model (`bench/run.py`, `src/engram/flags.py`). The existing-memories input to extraction diverges once the two stores diverge, so extraction state is not identical across arms: replayed from the cache, the two E2 arms gave extraction different existing memories on 69 of the 76 dev messages (from message D1:8 on), and 26 of the 76 messages produced different extraction outputs between the Jev and LLM arms

(`bench/e2_extraction_diff.py`). The E2 arms differ in what decides after extraction, and through it in the store that extraction reads.

Prompts. `mem0`’s extraction and update prompts, and the functions that build their requests, are copied verbatim from the installed `mem0 2.1.0` package (Apache License 2.0) into `src/engram/llm/prompts_mem0.py`. `tests/test_prompts_mem0.py` checks that the copies are byte-identical to the installed package, so a changed prompt fails the test suite. The answer and judge prompts are `mem0`’s LoCoMo evaluation prompts, adapted from two per-speaker memory lists to a single list (`bench/locomo_subset.py`).

4.3 Update sets

LoCoMo has almost no updates (§2.3). Two update sets, written in the speakers’ voices, extend it. Both are appended to the dev slice and were frozen by SHA-256 before any run.

Set 1 (`bench/updates_conv26.json`, SHA-256 prefix `2330876388f1`) has 30 items: 15 easy closes, 10 subtle items labeled *no_close* (past-tense mentions and unrealised plans that must not close anything), and 5 fulfilled plans. Set 2 (`bench/updates2_conv26.json`, prefix `d4f3c8d2de61`) adds 28 messages and 20 questions. Of these, 5 ask about a past moment in set 1’s items, 5 ask for the current value after a chain of 3–5 changes, 5 ask for a chain’s value at a past moment, and 5 follow an update whose message has no temporal cue such as “now” or “anymore”. Appendix B shows one item of each kind; the full sets are in the two files. Update sets 1–2 were drafted and labeled with an AI assistant (Claude) at the author’s direction; the author reviewed a subset. The 50 contradiction pairs were written and labeled by the author. This is a source of bias: the sets test failure modes the system’s builders anticipated.

4.4 Statistics

Comparisons are paired over the same questions (d_i , §3.1). We report an exact McNemar test on the discordant questions and a 95% interval for the mean of d_i from the per-question normal approximation. We also report a cluster bootstrap that resamples the four held-out conversations; with four clusters it is a robustness check, not a primary interval. Intervals are computed on the four scored categories, which stay the primary comparison.

5 Results

5.1 Replacing the decision layer

The three arms in Table 1 share extraction. The two engram arms differ only in what decides after it: Jev’s typed questions in E2 Jev, and claude-sonnet-4-6 with mem0’s update prompt, one call per extracted fact, in E2 LLM.

On the 35-question dev slice the two decision layers answer the same number of questions and mem0 one fewer, within noise. The Jev decision layer has $70.0\times$ lower decision cost and $27.6\times$ lower median decision latency than our claude-sonnet-4-6 implementation of mem0’s update prompt, one call per extracted fact. The LLM decision layer stores fewer facts (18 against 47) because mem0’s UPDATE event rewrites an existing memory instead of adding one.

The two latency columns are medians over different messages. In the E2 LLM arm only 30 of 76 messages made an LLM decision call, so the write median falls on an extraction-only message (median extraction 869 ms); on the messages with decisions, the logged median of the slowest decision call is 7,892 ms (bench/e2_latency.py).

In Table 1 the decision layer is 1.3% of the Jev arm’s end-to-end write cost and 47.0% of the LLM arm’s; with typed decisions, extraction is almost the whole cost of a write, and engram writes at about add-only mem0’s cost (\$9.90 against \$9.80 per 1,000 messages) although mem0 never re-examines what it stored.

Escalations and the frozen system. Both ratios exclude escalations: the E2 Jev arm made 0 on the dev slice. The frozen system (§3.2) escalates unsure superseding relations and asks more questions per fact; on the held-out conversations it escalated 31 decisions, its median decision latency was 868 ms to 1,982 ms per conversation, and its median end-to-end write was slower than mem0’s on 3 of 4 (Table 12, Appendix C). Its write cost stayed close to mem0’s (\$10.22 to \$10.55 against \$9.92 to \$10.16 per 1,000 messages).

The ratios are specific to that comparator: batching several facts per LLM call and a smaller LLM decider were not measured. On the stress slice the Jev arm scored 67/80 and mem0 64/80; the LLM arm was not run there.

At $k=20$ the two systems are indistinguishable. The held-out runs compare the frozen full system (E4 belief v2, §3.2) with mem0, not the E2 arms. At $k=20$ engram answers 482/610 and

mem0 477/610. The difference is +0.8 points (95% CI -2.3 to +3.9; conversation bootstrap -4.5 to +5.4; McNemar $p = 0.679$), within noise.

5.2 Retrieval under a small budget

Our answer-level result is consistent with the ranking-level improvement AtMem measured when reranking with Jev (Taghia, 2026).

At $k=3$ engram shows the answer model $T(3) = 290$ tokens per question and mem0 159. Most of the difference is the source quote on each engram line. To separate context size from ranking, we answered the same 610 questions from mem0’s existing held-out stores at every k from 3 to 8. The token-matched setting is the k whose mean retrieved tokens came closest to engram’s 290: $k=6$, at 315 tokens. That gives mem0 slightly more context than engram. The run cost \$2.52.

Table 2 has the pooled results. At $k=3$ engram is ahead of mem0 by +14.9 points (95% CI +11.2 to +18.7). At a matched mean retrieved-context budget, engram was +8.7 points above mem0 (95% CI +5.2 to +12.1; conversation bootstrap +2.4 to +14.7; McNemar p -value $1.3e-06$; 86 questions only engram answered against 33 only mem0 answered). Figure 2 places every measured setting on one axis of $T(k)$.

engram’s point estimate is ahead of token-matched mem0 in each of the four scored categories (Table 3) and in every conversation (Appendix C). Per conversation, the matched-context interval excludes zero in 2 of four conversations (conv-42 and conv-43); conv-30 and conv-41 are within noise on their own.

Table 3 adds LoCoMo’s adversarial category, 190 questions about things the conversations never say, whose gold answer is to abstain. Our intervals are computed on the four scored categories, which stay the primary comparison. engram with reranking scores lowest on adversarial questions at both budgets (71.1% at $k=3$, 67.9% at $k=20$, against 75.8% for token-matched mem0 and 76.3% with reranking off); over all five categories engram $k=3$ scores 72.8% and token-matched mem0 67.2%. We did not test why; the answer prompt does not ask for abstention, and a context of relevant memories may invite an answer.

Attribution. Matching context removes 41.8% of the $k=3$ difference. What remains is a question of which memories are shown, which a held-out arm with Jev’s reranking off answers (Table 2). From the same frozen stores, with the same an-

Table 1: Dev slice, conv-26 sessions 1–4, 76 messages, 35 questions, all retrieved memories (default k). Extraction claude-haiku-4-5 for all arms; answers and judge claude-sonnet-4-6; E2 LLM decides with claude-sonnet-4-6. Costs are per 1,000 messages written. Decision p50: median time after extraction, per message, over messages that produced at least one fact. Write p50: median end-to-end write time per message over all messages, including those that produced no facts. The two decision layers answer the same number of questions, while the Jev layer has $70.0\times$ lower decision cost and $27.6\times$ lower median decision latency than our claude-sonnet-4-6 implementation of mem0’s update prompt, one call per extracted fact.

system	accuracy	decision \$/1k	decision p50	total \$/1k	write p50	facts
mem0 2.1.0 (one LLM call per write)	30/35	–	–	\$9.80	879 ms	46
engram E2, LLM decision layer (mem0 update prompt)	31/35	\$8.782	7,675 ms	\$18.70	918 ms	18
engram E2, Jev decision layer	31/35	\$0.125	278 ms	\$9.90	895 ms	47

Table 2: Held-out LoCoMo accuracy, pooled over conv-30, 41, 42 and 43 (610 questions, adversarial category excluded). Each row after an engram row carries its paired difference against engram at the same budget (engram $k=3$ for the $k=3$, reranking-off and token-matched rows, engram $k=20$ for the $k=20$ row); Δ is engram minus the row. Extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6; one ingestion per system per conversation. Tokens: $T(k)$. 95% CI: per-question normal approximation; bootstrap: resampling the four conversations. Discordant: questions only the engram row / only this row answered correctly. engram leads token-matched mem0 at $k=3$, turning reranking off costs more than that lead, and at $k=20$ the two systems are indistinguishable.

system	k	tokens/q	accuracy	Δ (pts)	95% CI	95% CI, bootstrap	discordant	McNemar p
engram	3	290	73.3%	–	–	–	–	–
mem0	3	159	58.4%	+14.9	[+11.2, +18.7]	[+8.2, +19.8]	120 / 29	2.4e-14
engram, reranking off	3	282	59.8%	+13.4	[+10.1, +16.8]	[+8.7, +17.8]	101 / 19	1.1e-14
mem0 (token-matched)	6	315	64.6%	+8.7	[+5.2, +12.1]	[+2.4, +14.7]	86 / 33	1.3e-06
engram	20	1,461	79.0%	–	–	–	–	–
mem0	20	1,024	78.2%	+0.8	[-2.3, +3.9]	[-4.5, +5.4]	49 / 44	0.679

swer and judge models, reranking off shows the answer model $T(3) = 282$ tokens, close to the full pipeline’s, and answers 365/610 (59.8%). The full pipeline answers 13.4 points more (95% CI +10.1 to +16.8; conversation bootstrap +8.7 to +17.8; McNemar p-value $1.1e-14$), and reranking off is -4.8 points against token-matched mem0 (95% CI -8.4 to -1.1; McNemar p-value 0.014). On held-out data the reranker therefore accounts for the whole matched-context lead; without it, the rest of the pipeline (floor, relation pull, history, rendering) is below mem0 at the same budget. With reranking off the answer model sees the cosine top three Eq. (19). The store copies reproduced the frozen run’s $k=3$ context for all but 3 of the 610 questions (bench/heldout_extra.py).

5.3 Store behaviour and calibration

Table 4 lists, for each arm, its closes, over-closes and stale items together with its accuracy on the update questions, and Figure 4 plots the store outcomes.

Trap items. In the frozen arm 0/8 no-close trap items were closed, with 1 wrong plan_fulfilled close and 1 close matching no labeled pair; no

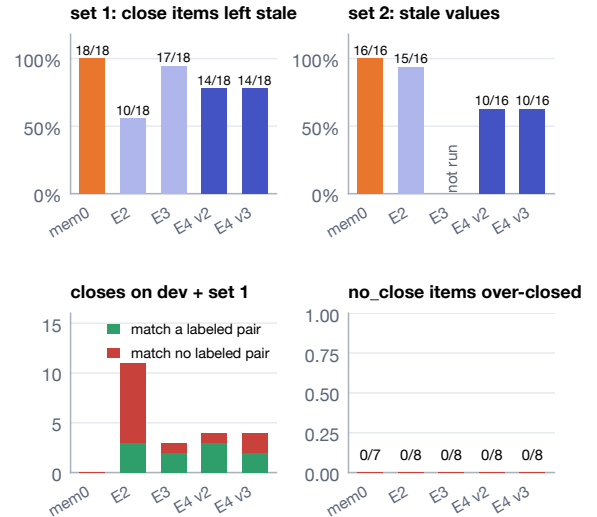


Figure 4: Store outcomes on the update sets: set-1 close items left stale, set-2 stale values, closes on dev + set 1 split by whether they match a labeled pair, and no_close items over-closed. E3 was not run on set 2. The replace-on-update arm (E2) makes most of the closes that match no labeled pair; the belief arms close less and leave more set-1 items stale, and mem0 closes nothing.

Table 3: LoCoMo, four held-out conversations, LLM-as-a-judge with mem0’s judge prompt on claude-sonnet-4-6; extraction claude-haiku-4-5, answers claude-sonnet-4-6. Adversarial gold answers require abstention, which mem0’s answer prompt does not instruct; both systems share that handicap. Scores are not comparable to leaderboards run on other model stacks. engram at $k=3$ is ahead of token-matched mem0 in all four scored categories and behind every mem0 setting on adversarial questions.

method	Multi-Hop	Temporal	Open-Domain	Single-Hop	Adversarial	Overall
questions	110	119	33	348	190	800
engram $k=3$	49.1%	84.0%	51.5%	79.3%	71.1%	72.8%
engram $k=3$, reranking off	31.8%	75.6%	33.3%	65.8%	76.3%	63.7%
mem0 $k=3$	31.8%	71.4%	30.3%	64.9%	75.3%	62.4%
mem0 $k=6$ (token-matched)	40.9%	73.9%	33.3%	71.8%	75.8%	67.2%
engram $k=20$	67.3%	88.2%	51.5%	82.2%	67.9%	76.4%
mem0 $k=20$	61.8%	84.0%	57.6%	83.3%	75.3%	77.5%

Table 4: Store outcomes and update-question accuracy on the dev slice with update sets 1 and 2 (extraction claude-haiku-4-5, decisions Jev jev-1.13.0, answers and judge claude-sonnet-4-6). Closes are on dev + set 1, split by whether (closed fact’s message, closing message) is a labeled update or superseded pair. Over-closed: set-1 no_close items whose fact was closed by their own update message, over those stored. Stale: close items whose old fact is still active, over close items stored. Every arm kept the one set-2 keep item. Accuracy is at default k (all retrieved memories) unless marked; “–”: run without answering or not run. No arm closed a no-close trap item (0/8 for every engram arm, 0/7 for mem0), and closing stale facts lowers the stale counts without moving update-question accuracy at default k , which is at or near ceiling for every arm that answered.

arm	closes	labeled	unlabeled	over-closed	set-1 stale	set-2 stale	set-1 acc.	set-1 acc. $k=3$	set-1 acc. no dates	set-2 acc.
mem0 (add-only)	0	0	0	0/7	18/18	16/16	29/30	25/30	29/30	20/20
E2: Jev, replace-on-update	11	3	8	0/8	10/18	15/16	30/30	30/30	30/30	19/20
E3: structural rules	3	2	1	0/8	17/18	not run	–	–	–	–
E4: belief v1	3	2	1	0/8	17/18	15/16	30/30	30/30	29/30	19/20
E4: belief v2 (frozen)	4	3	1	0/8	14/18	10/16	30/30	30/30	–	20/20
E4: belief v3	4	2	2	0/8	14/18	10/16	–	–	–	–

other arm closed a trap item either.

Closes outside the labels. These separate the policies. The first Jev arm (E2), which closed whenever a superseding label cleared 0.85, made 11 closes on dev + set 1, and 8 of them match no labeled pair. Belief v2 made 4, of which 1 match no labeled pair. On set 2 the comparison is 10 of 14 against 1 of 10.

Stale items. A stale item is a close item whose old fact is still active. Belief v2 left 14/18 of the set-1 close items stale, against 17/18 under belief v1 and 18/18 for mem0, which never closes. On set 2 it left 10/16 stale, against 15/16 and 16/16.

Fulfilled plans. A relaxed heuristic closed a plan whenever a related past or current fact arrived. Across its two arms it fired 25 times. 1 firing matched a labeled (plan, fulfilment) pair, and 24 did not. The frozen arm uses the plan_fulfilled question instead; it was asked 234 times on dev + set 1 and closed 2 plans correctly and 1 wrongly.

Belief v3. On the same data, v3 ignored 36 weak answers on dev + set 1 and 39 with set 2 added. It

closed the same four facts as v2 on dev + set 1. One of them, D2:5, crossed the close line at a different message (U:S01 rather than U:E11). Since that pair is not labeled, the audit scores v3 at 2 matching and 2 not matching, against v2’s 3 and 1. Stale and over-close counts are unchanged. v3 targets the weak-support failure that closed true facts under Laya (Appendix D, exploratory).

Hygiene. One pass cost \$0.0015 on dev + set 1 (235 decisions). On the held-out conversations it cost \$0.0112, \$0.0254, \$0.0332 and \$0.0323, making 9, 23, 32 and 24 links.

Contradiction regression. Table 14 (Appendix D) scores Jev on the regression set, next to Laya. The set has 50 contradiction pairs, each an old fact, a new fact and a message, labeled with accepted relations and a temporal status (bench/contradiction_pairs.jsonl). Under the current question versions, Jev’s relation choice is in the accepted set for 90.0% of pairs (easy 100.0%, medium 93.3%, subtle 73.3%), and its temporal status is right for 94.0%. The write path’s close

rule is right for 76.0%, with 0 false closes. Jev’s mean top probability is 0.88 when it is right and 0.81 when it is wrong.

Calibration. We measure calibration on two label sets: the 50 gold pairs, and 29 escalation labels, where Sonnet decided a relation Jev was unsure of. At these sizes expected calibration error (ECE) is exploratory, so Table 5 also reports the Brier score and negative log-likelihood (NLL), both with bootstrap 95% intervals. On the gold pairs Jev’s relation NLL is 0.57 (0.29 to 0.89) and its relation ECE 0.14 (fitted $\tau_Q = 1.48$). On the escalation labels Jev’s relation NLL is 3.18 (0.38 to 6.43): its mean confidence there is 0.92, so the relations it gets wrong it gets wrong with near-certain probability. Figure 6 in Appendix C has the reliability diagrams.

Cost/error tradeoff. The 29 escalation labels are too few for a curve, so Figure 5 uses the 50 gold pairs. The Jev cost $c_J = \$0.000016$ is the mean cost of one relation decision over 36,429 held-out decisions, and the escalation cost $c_L = \$0.0073$ is the mean over 148 logged escalations. No LLM run on the gold pairs is saved, so ε_L is an assumption, plotted at 0 and 0.1. With Jev alone the error is 10.0%. At $\theta = 0.85$, 32.0% of decisions escalate, the cost is \$0.002368 per decision, and E is 6.0% ($\varepsilon_L = 0$) or 9.2% ($\varepsilon_L = 0.1$). At the production threshold of 0.60, 8.0% escalate and E is 10.0% or 10.8%.

5.4 Negative result: closes do not change answers

Closes do not change answers. The accuracy columns of Table 4 show it. mem0 never closes anything and leaves every set-1 close item stale (18/18), yet answers 29/30 set-1 questions at default k . The arm that closes most aggressively leaves 10/18 stale and answers 30/30.

The answer model resolves recency itself: extraction writes dates into the memory text, and the compact rendering adds the date each fact was said. Removing all dates, validity and source from the answer context (the no-dates column) still leaves every arm we ran that way at 29/30 or higher on set 1. Only the $k=3$ budget separates the systems (25/30 for mem0, 30/30 for engram), which is a retrieval effect (§5.2), not a close effect.

Set 2, which asks about chains of changes and past moments, is at its ceiling for every arm. Current LoCoMo-style questions do not reward a correct store.



Figure 5: $C(\theta)$ and $E(\theta)$ on the 50 gold pairs. ε_L is assumed, not measured; the 29 escalation labels were too few for a curve. Error drops below Jev’s own 10.0% only where escalations make up almost all of the cost per decision, and how far it drops depends on an ε_L we did not measure.

5.5 Systems notes

Jev latency varies modestly with request size below 50 questions; run-to-run variation is larger.

Across 9,446 live Jev requests from 25 runs, median latency is between 222 ms and 269 ms for requests of 1 to 50 questions, and 371 ms for 51–80. A least-squares fit gives 380 ms plus 7.19 ms per question (Figure 7 and Table 11, Appendix C). For requests of 16–20 questions, the per-run median was between 212 ms and 267 ms in 12 runs, and 793 ms and 607 ms in the two held-out runs made during one slower period.

Extraction dominates write cost. On the held-out set, engram’s write cost is \$10.22 to \$10.55 per 1,000 messages and mem0’s is \$9.92 to \$10.16. The decision layer is 3.3% to 4.8% of engram’s (Table 12, Appendix C).

Read path. Per held-out question, engram’s read path makes one Jev request, which scores every shortlisted fact and classifies the question’s relation. It cost \$0.00039 per question on average, and its median latency per conversation was between 292 ms and 2,531 ms, following the run-to-run variation above (Table 13, Appendix C). mem0’s read path is a local embedding and vector search with no API call; its latency was not logged. engram’s longer memory lines also make the answer call dearer:

Table 5: Calibration of Jev (jev-1.13.0) on the escalation labels (dev + update sets, labels by claude-sonnet-4-6) and the gold contradiction pairs. ECE (exploratory at these n) uses 10 equal-width bins on the top choice; τ_Q is fitted by NLL per question and “ECE at τ_Q ” is out of sample (2-fold). Brier: sum over options of the squared error, averaged over items; NLL: $-\ln$ of the probability on the label; intervals: 10,000 bootstrap resamples of the items. Relation probabilities fold negatives into contradiction, since the pairs predate negatives. Laya’s rows are in Table 15 (Appendix D). Jev’s relation accuracy is about 80% on both label sets; on the escalation labels its errors are confident, which raises its NLL.

labels	question	backend	n	top-1 acc.	mean conf.	ECE	fitted τ	ECE at τ	Brier [95% CI]	NLL [95% CI]
escalation	relation	jev	29	79.3%	0.92	0.15	2.51	0.10	0.34 [0.12, 0.61]	3.18 [0.38, 6.43]
gold pairs	relation	jev	50	82.0%	0.87	0.14	1.48	0.14	0.30 [0.16, 0.46]	0.57 [0.29, 0.89]
gold pairs	temporal status	jev	50	94.0%	0.95	0.04	1.13	0.04	0.08 [0.02, 0.17]	0.14 [0.04, 0.31]

\$0.00245 per question against \$0.00229 for mem0 at $k=3$.

6 Discussion

What the decision layer buys, and what it does not. Typed decisions buy lower cost and latency (§5.1), an audit trail in which every decision carries probabilities and a question version, store operations that are reversible and gated (§5.3), and a price at which re-examining the store is routine (§3.2). They do not buy accuracy at $k=20$, where the two systems are indistinguishable (§5.1), or accuracy on these update questions, where every arm is near its ceiling (§5.4).

Benchmarks do not see storage correctness.

A question about a fact that changed is answerable from a store that kept both versions, as long as the memories carry dates. An evaluation that rewards a correct store would score answers against validity windows, use small retrieval budgets in which a stale fact displaces a current one, store memories without dates, and ask about long chains of changes. Update set 2 targets the last of these, and every arm is at its ceiling on it.

The reranker, not the context size. Under a three-memory budget engram leads token-matched mem0 by +8.7 points, and turning Jev’s reranking off removes that lead and more (13.4 points). The two systems are indistinguishable at $k=20$. On adversarial questions the reranked context scores lowest (Table 3); an answer prompt that asks for abstention was not tested.

7 Conclusion

Typed decisions let engram re-examine its store at about the write cost of add-only mem0. Against an LLM decider using the same extraction model, prompt construction and implementation, they had

$70.0\times$ lower decision cost and $27.6\times$ lower median decision latency, with equal accuracy on a 35-question dev slice. On 610 held-out questions, at a matched mean retrieved-context budget engram was +8.7 points above mem0; turning Jev’s reranking off removed that lead and left engram at -4.8 points against mem0, so the reranker accounts for it. At $k=20$ the two systems are indistinguishable. Closing stale facts, the part of the design aimed at correctness, did not change answers on our update sets: current LoCoMo-style questions do not reward a correct store.

Limitations

We make no claim beyond these LoCoMo-derived evaluations with this extraction, rendering and answer setup. The comparison has one baseline, mem0 OSS 2.1.0, on one benchmark, four held-out LoCoMo conversations. Update sets 1–2 were drafted and labeled with an AI assistant (Claude) at the author’s direction; the author reviewed a subset. The 50 contradiction pairs were written and labeled by the author.

A closed, versioned API. Every decision comes from Jev jev-1.13.0, served by TypeSafe’s hosted API; its weights are not public. A later version may decide differently, and if this version is withdrawn our results can be replayed from the call cache but not recomputed. Jev is the only decision model evaluated as the deciding backend, and the open-weights Laya was tested only as a zero-shot base checkpoint (Appendix D).

The judge is the answer model. claude-sonnet-4-6 both answers and judges, so the judge may favour answers written the way it would write them, and we did not measure its agreement with human labels.

Extraction shares model, prompt construction and implementation but not state: the stores it reads

diverge (§4.2), and no arm froze extraction outputs across decision layers. mem0’s open-source path gives no observation date, so relative dates resolve against the run date; we use it as shipped, and a variant with the session date patched in scored 31/35 on dev against 30/35 unpatched, within noise. Adversarial questions require abstention, which mem0’s answer prompt does not ask for (Table 3).

AI Assistance

Code, experiment orchestration and paper drafting were carried out with Claude (Anthropic) via Claude Code under the author’s direction; the author designed the study, made every methodological decision, and is responsible for all claims.

References

- Anastasios N. Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*.
- ByteRover. 2026. Benchmarking AI agent memory: ByteRover 2.0 scores 92.2% and rewrites the LoCoMo leaderboard. <https://byterover.dev/blog/benchmark-ai-agent-memory>. Accessed September 2026.
- Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*. ArXiv:2305.05176.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Convai Innovations. 2026. Laya: a non-autoregressive system 1 decision model. Hugging Face model card, <https://huggingface.co/convaiinnovations/laya>. ModernBERT-large, 421M parameters. Accessed September 2026.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. ArXiv:1706.04599.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*.
- Dongming Jiang, Yi Li, and Bingzhe Li. 2026a. Jev-Mem: System-one-controlled agentic memory for efficient AI agents. *arXiv preprint arXiv:2609.23986*. Concurrent work, posted 21 September 2026.
- Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. 2026b. MAGMA: A multi-graph based agentic memory architecture for AI agents. *arXiv preprint arXiv:2601.03236*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. ArXiv:2307.03172.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, Bangkok, Thailand. Association for Computational Linguistics. ArXiv:2402.17753.
- Mem0. 2026. LoCoMo vs. LongMemEval vs. BEAM: The 2026 AI memory benchmark guide. <https://mem0.ai/blog/ai-memory-benchmarks-in-2026>. Accessed September 2026.
- Andy Nguyen, Danh Doan, Hoang Pham, Bao Ha, Dat Pham, Linh Nguyen, Hieu Nguyen, Thien Nguyen, Cuong Do, Phat Nguyen, and Toan Nguyen. 2026. ByteRover: Agent-native memory through LLM-curated hierarchical context. *arXiv preprint arXiv:2604.01599*.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to route LLMs with preference data. *International Conference on Learning Representations (ICLR)*. ArXiv:2406.18665.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*. The system now maintained as Letta.
- Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of EMNLP*. ArXiv:2304.09542.
- Javad Taghia. 2026. Governed agent memory with structured judgment: An AtMem-Jev retrieval study. Hugging Face community article, <https://huggingface>

[e.co/blog/javadtaghia/an-atmem-and-jev-experiment-can-a-judgment-model-h](https://javadtaghia.com/blog/javadtaghia/an-atmem-and-jev-experiment-can-a-judgment-model-h). Published 19 September 2026.

TypeSafe AI. 2026. Jev: introduction and the choice primitive. <https://docs.typesafe.ai/introduction>. Accessed September 2026.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*. ArXiv:2410.10813.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-mem: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.

A Decision questions

Table 6 lists the 12 questions in the current chain with their types and options. `edge_type` and `query_relation` choose among 25 relation types. The instructions and the rubric for every option of every version are in `src/engram/decide/questions.py`, and the reasons for each version change are in `docs/DECISIONS.md`.

Table 6: Jev questions in the current chain (`src/engram/decide/questions.py`).

question	type	options
worth_remembering	noul v1	yes / no
fact_kind	choice v1	preference, bio, event, relationship, task, opinion
temporal_status	choice v2	current, planned, past, hypothetical
relation_to_candidate	choice v2	new, duplicate, update, contradiction, refinement, negates
relation_to_candidate_recheck	choice v2	new, duplicate, update, contradiction, refinement, negates
edge_type	choice v1	25 relation types
durability	choice v1	permanent, long_term, short_lived
sensitivity	choice v1	none, health, financial, relationship, credentials
plan_fulfilled	noul v1	yes / no
relevant_to_query	noul v1	yes / no
query_relation	choice v1	25 relation types
same_fact	noul v1	yes / no

B Update sets

Table 7: One item of each kind from the two update sets (`bench/updates_conv26.json`, `bench/updates2_conv26.json`).

id	kind	earlier fact or message	update message(s)	question	gold
E01	set 1, easy close	Melanie’s main creative outlet is painting, which relaxes her after a long day.	I haven’t painted in weeks, honestly. Pottery is my main creative outlet now - I go to the studio three evenings a week.	What is Melanie’s main creative outlet now?	Pottery (she has stopped painting)
S01	set 1, subtle (no_close)	Melanie has been married to her husband for 5 years.	Our first apartment after the wedding was so tiny - we still laugh about it.	Is Melanie still married?	Yes, married for 5 years
F01	set 1, fulfilled plan	Caroline plans to continue her education.	I did it - I enrolled in a psychology certificate program at the community college. Classes start next week!	Has Caroline enrolled in a program to continue her education?	Yes, a psychology certificate program at the community college
C01q1	set 2, chain	I just moved into a studio apartment downtown.	Moved again! I’m in a two-bedroom in Oak Park now, so there’s room for a foster kid. → We finally found a house - I live in Evanston now.	Where does Caroline live now?	A house in Evanston
N01	set 2, no temporal cue	My favorite coffee spot is Blue Door Café.	My favorite coffee spot is Grind House.	What is Caroline’s favorite coffee spot?	Grind House

C Per-conversation and systems tables

Table 8: Held-out accuracy per conversation, $k=3$. Q per row; models as in Table 2.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	59/81	55/81	4	285	154
conv-41	152	113/152	89/152	24	297	161
conv-42	199	150/199	111/199	39	292	163
conv-43	178	125/178	101/178	24	283	153

Table 9: Held-out accuracy per conversation, engram $k=3$ against token-matched mem0 ($k=6$). Q per row; models as in Table 2.

conv.	Q	engram $k=3$	mem0 $k=6$	Δ (questions)
conv-30	81	59/81	54/81	5
conv-41	152	113/152	109/152	4
conv-42	199	150/199	119/199	31
conv-43	178	125/178	112/178	13

Table 10: Held-out accuracy per conversation, $k=20$. Q per row; models as in Table 2.

conv.	Q	engram	mem0	Δ (questions)	engram tokens / q	mem0 tokens / q
conv-30	81	62/81	63/81	-1	1,434	996
conv-41	152	128/152	135/152	-7	1,521	1,049
conv-42	199	156/199	146/199	10	1,491	1,043
conv-43	178	136/178	133/178	3	1,389	995

Table 11: Jev (jev-1.13.0) latency by request size over all logged runs (dev, stress and held-out slices), from the decision logs (bench/results/jev_latency.json). Client-measured, after the rate limiter, retries included.

questions	requests	input tokens	p50	p90
1	1,305	564	229 ms	1,052 ms
2-3	1,054	870	248 ms	1,176 ms
4-6	775	1,562	231 ms	992 ms
7-10	1,417	4,314	246 ms	967 ms
11-15	227	2,432	231 ms	680 ms
16-20	3,240	5,593	255 ms	1,009 ms
21-30	184	3,551	222 ms	312 ms
31-50	505	6,903	269 ms	472 ms
51-80	739	9,187	371 ms	2,290 ms

Table 12: Held-out write side (conv-30, 41, 42, 43; writes only). Extraction claude-haiku-4-5 for both systems; decisions Jev jev-1.13.0 with claude-sonnet-4-6 escalations. One ingestion per system per conversation. Decision layer = Jev plus escalations.

conv.	engram \$/1k	decision \$/1k	mem0 \$/1k	decision p50	engram write p50	mem0 write p50	engram facts	mem0 memories
conv-30	\$10.22	\$0.33	\$9.92	1,982 ms	928 ms	1,034 ms	267	276
conv-41	\$10.54	\$0.47	\$10.16	1,614 ms	1,775 ms	1,437 ms	668	822
conv-42	\$10.55	\$0.50	\$10.04	981 ms	1,824 ms	1,300 ms	708	695
conv-43	\$10.45	\$0.50	\$9.96	868 ms	1,819 ms	1,210 ms	628	636

Table 13: Held-out read path per question (conv-30, 41, 42, 43; $k=3$), from the frozen runs' decision logs (bench/results/read_path.json). engram: one Jev request per question, client-measured latency; mem0's read path is a local embedding and vector search with no API call, and its latency was not logged. Answer: claude-sonnet-4-6.

conv.	questions	engram read \$/query (Jev)	engram read p50	engram read p90	engram answer \$/query	mem0 answer \$/query
conv-30	81	\$0.00038	2,531 ms	4,822 ms	\$0.00240	\$0.00227
conv-41	152	\$0.00040	292 ms	428 ms	\$0.00245	\$0.00228
conv-42	199	\$0.00039	1,984 ms	2,384 ms	\$0.00249	\$0.00232
conv-43	178	\$0.00039	307 ms	475 ms	\$0.00242	\$0.00228

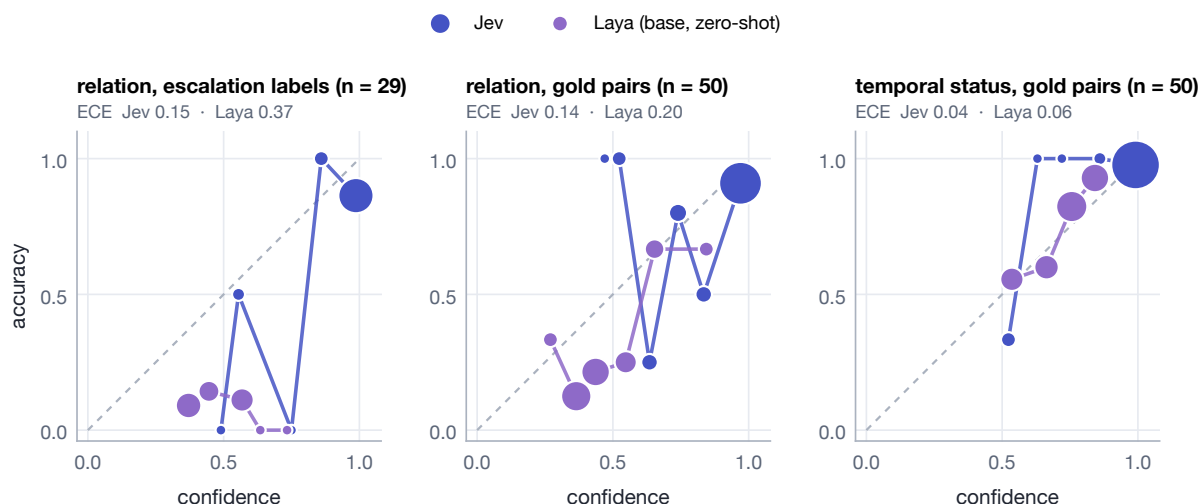


Figure 6: reliability diagrams (accuracy against confidence) for Jev and Laya on both label sets.

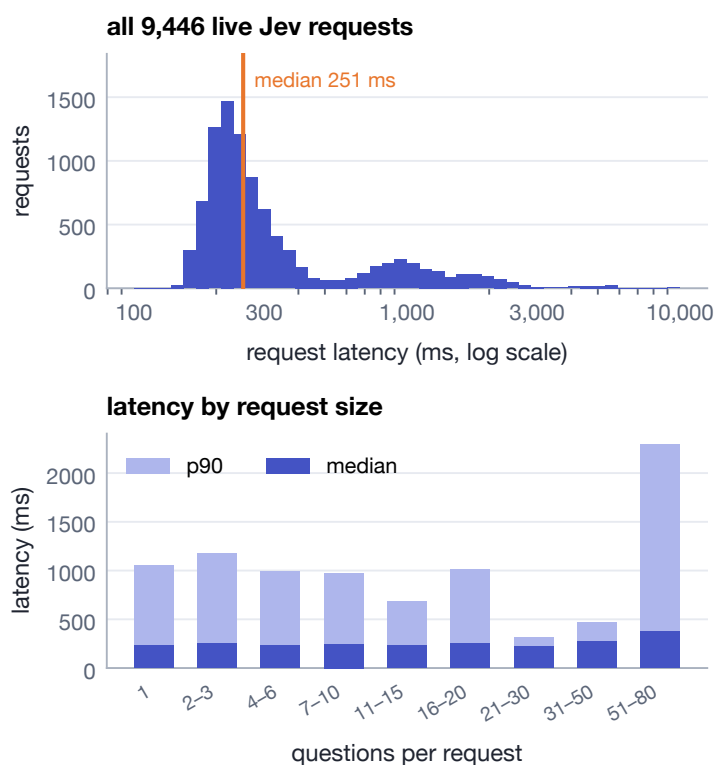


Figure 7: Jev request latency: distribution over all logged requests, and median and p90 by request size.

D Laya, an open-weights checkpoint (exploratory)

Laya. Laya is an open-weights model with the same request format. We ran checkpoint convaiinnovations/laya locally through the laya-mlx port on an Apple M2 Pro with 32 GB. It reads at most 512 tokens per question, of which at most 192 go to the instructions and options. This is the 421M base checkpoint. Its model card reports zero-shot typed-decision accuracy of 0.362 against a 0.318 random baseline, calls it “a fast base to specialise, not a zero-shot decision engine,” and offers a separate checkpoint fine-tuned for typed decisions (Convai Innovations, 2026). We used the base checkpoint zero-shot.

Everything in this part concerns the 421M base checkpoint, used zero-shot as its documentation advises against (Convai Innovations, 2026). The checkpoint fine-tuned for typed decisions (reported at 0.766 on its own benchmark) and fine-tuning on our escalation labels were not tested; they are the obvious

follow-up.

On the 50 regression pairs (Table 14), Laya chooses an accepted relation for 48.0% with Jev’s question wording and 38.0% with wording rewritten to fit its 192-token question budget. Jev scores 90.0%. Laya never reaches the action threshold, so it closes nothing. Its gold-pair relation accuracy is 28.0% (Table 15). A temperature lowers its ECE but not its accuracy.

With the frozen Jev arm replayed from cache and Laya answering every request on the side, the two gave the same answer on 44.8% of 6,764 decisions and the same action at the 0.85 threshold on 31.2%. On relation_to_candidate they agreed on 5.9% (Table 16 and Figure 8).

When Laya decided every question, on dev + set 1 at $k=3$ it scored 24/35 LoCoMo and 26/30 update questions. Jev scored 29/35 and 30/30. It made 23 closes, of which 23 match no labeled pair, and left 43 of 66 facts active. Most of these closes came from weak duplicate and refinement answers below 0.5 lowering belief, the failure belief v3 removes Eq. (10).

Hybrid. In a hybrid arm Laya answered the two high-volume yes/no questions (relevant_to_query, same_fact) and Jev the rest. On the routed questions the two models gave the same answer on 86.9% and the same action on 34.1%. Writes were identical to the all-Jev arm, but at $k=3$ only 51.8% of Jev’s top-3 lines survived the change of reranker. The hybrid saved 42.6% of Jev’s cost on dev + set 1 (\$0.025 per run); Table 17 has its accuracy.

Table 14: Contradiction regression (50 gold pairs), relation_to_candidate and temporal_status in one request. Laya: convaiinnovations/laya base checkpoint, zero-shot, fp16, on the local MLX server. Jev chooses an accepted relation for 90.0% of the pairs with no false close; the Laya base checkpoint chooses one for at most 48.0% and never closes.

backend	relation exact	temporal	close rule	closes	false closes
Jev (jev-1.13.0)	90.0%	94.0%	76.0%	22	0
Laya, Jev wording (truncated)	48.0%	76.0%	32.0%	0	0
Laya, native wording (fits)	38.0%	76.0%	32.0%	0	0

Table 15: Calibration of the Laya base checkpoint (zero-shot) on the same label sets and with the same measures as Table 5.

labels	question	backend	n	top-1 acc.	mean conf.	ECE	fitted τ	ECE at τ	Brier [95% CI]	NLL [95% CI]
escalation	relation	laya	29	10.3%	0.47	0.37	5.41	0.12	1.00 [0.90, 1.09]	2.11 [1.83, 2.41]
gold pairs	relation	laya	50	28.0%	0.47	0.20	4.22	0.02	0.83 [0.72, 0.94]	1.81 [1.53, 2.09]
gold pairs	temporal status	laya	50	76.0%	0.72	0.06	0.74	0.07	0.33 [0.22, 0.45]	0.57 [0.44, 0.72]

Table 16: Laya against Jev on identical requests: dev + update sets 1 and 2, frozen arm’s trajectory at $k=3$ (write and retrieval decisions; n per row). Jev jev-1.13.0; Laya base checkpoint, zero-shot. Same action: both choose the same label at $\pi \geq 0.85$, or neither reaches it. Acts: share of decisions at $\pi \geq 0.85$.

question	n	same answer	same action	Jev acts	Laya acts
relation_to_candidate	2,779	5.9%	18.5%	79.9%	6.4%
relevant_to_query	1,440	93.5%	38.9%	89.7%	31.3%
same_fact	585	72.6%	35.2%	96.2%	34.4%
plan_fulfilled	522	66.5%	16.1%	90.0%	32.6%
worth_remembering	226	53.5%	77.0%	18.1%	4.9%
fact_kind	226	41.6%	38.1%	63.7%	15.5%
temporal_status	226	66.8%	42.5%	62.8%	5.3%
edge_type	226	41.6%	50.9%	29.2%	55.3%
durability	226	43.8%	47.3%	52.7%	0.0%
sensitivity	226	66.4%	57.5%	45.1%	7.1%
query_relation	48	54.2%	47.9%	39.6%	75.0%
relation_to_candidate_recheck	34	29.4%	41.2%	35.3%	26.5%

Table 17: Hybrid against all-Jev: dev slice with update sets (Q per row: 35 LoCoMo, 30 set 1, 20 set 2), k as labeled, extraction claude-haiku-4-5, answers and judge claude-sonnet-4-6.

questions	all-Jev $k=3$	hybrid $k=3$	all-Jev $k=20$	hybrid $k=20$
update set 1	30/30	27/30	30/30	30/30
LoCoMo dev	29/35	29/35	31/35	31/35
update set 2	19/20	17/20	20/20	20/20

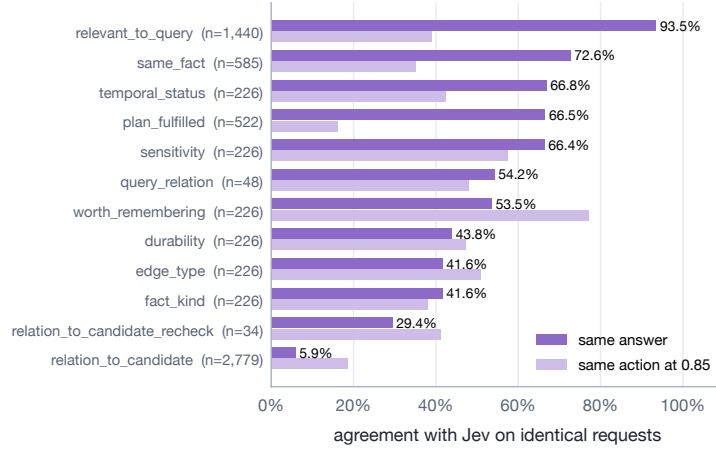


Figure 8: Agreement between the Laya base checkpoint (zero-shot) and Jev on identical requests, per question, sorted by same answer. Dev + update sets 1 and 2, frozen arm’s trajectory.

E Reproduction

Each table and figure was produced by the command below, run from the root of the engram codebase. With the call cache (bench/.cache/calls.sqlite) in place, every command in Table 18 replays at no API cost.

Table 18: The command behind each table and figure.

table / figure	command
Table 1	python -m bench.run --arm {e2_jev,e2_llm,mem0} --slice dev
Tables 2, 3, 8–10; Fig. 2	python -m bench.run --arm {e4_belief_v2,mem0} --slice heldout:<conv> --top-k 3 --also-top-k 20, then python -m bench.heldout_report, python -m bench.token_match, python -m bench.perconv and python -m bench.heldout_extra (reranking off, adversarial)
§4.2	python -m bench.e2_extraction_diff
Table 4; Figs. 3, 4	python -m bench.run --arm <arm> --slice dev_updates[2] [--top-k 3] [--no-dates]
Tables 5, 14, 15; Figs. 6, 8	python bench/test_contradictions.py --backend laya [--native] --save . . . , python -m bench.calibration, python -m bench.calibration_scores, python -m bench.jev_regression (the Laya rows need bench/laya_server.py running)
Fig. 5	python -m bench.tradeoff
Tables 16, 17	python -m bench.laya_report, python -m bench.hybrid_report
Table 11, Fig. 7	python -m bench.jev_latency
Table 13; §5.5 read path	python -m bench.read_path
this paper	python paper/build.py, python paper/figures.py

The spend ledger records cost per run, not per table; Table 19 gives per-arm totals.

Table 19: Phase 2 spend per arm, in USD (bench/results/phase2_spend.jsonl).

arm	runs	Jev	Claude
mem0	15	\$0.0000	\$35.94
e4_belief_v2	18	\$1.0917	\$31.80
heldout_extra	4	\$0.0365	\$8.50
e2_jev	2	\$0.0636	\$3.51
e2_llm	2	\$0.0134	\$3.21
mem0_token_matched	1	\$0.0000	\$2.52
e4_belief_v2_laya	2	\$0.0405	\$2.40
e2_jev_v2	4	\$0.0362	\$2.35
e1_recall	4	\$0.0653	\$2.05
e4_belief	6	\$0.0316	\$1.69
e3_structural_v2	1	\$0.0278	\$1.44
e0_baseline	8	\$0.0736	\$1.36
mem0_dated	1	\$0.0000	\$0.96

arm	runs	Jev	Claude
e3_structural_v3	2	\$0.0272	\$0.76
e4_belief_v2_hybrid	4	\$0.0000	\$0.73
e4_belief_v2_compact	2	\$0.0000	\$0.59
e4_belief_v2_norerank	1	\$0.0000	\$0.25
e4_belief_v2_shadow	2	\$0.0000	\$0.18
e3_structural	1	\$0.0008	\$0.04
e4_belief_v2_nohist	1	\$0.0000	\$0.01
e4_belief_v3	3	\$0.0004	\$0.00